

Transport protocols and application performance

Toke Høiland-Jørgensen - Network Services Team
Simone Ferlin-Reiter - Perf & Scale Team

Agenda

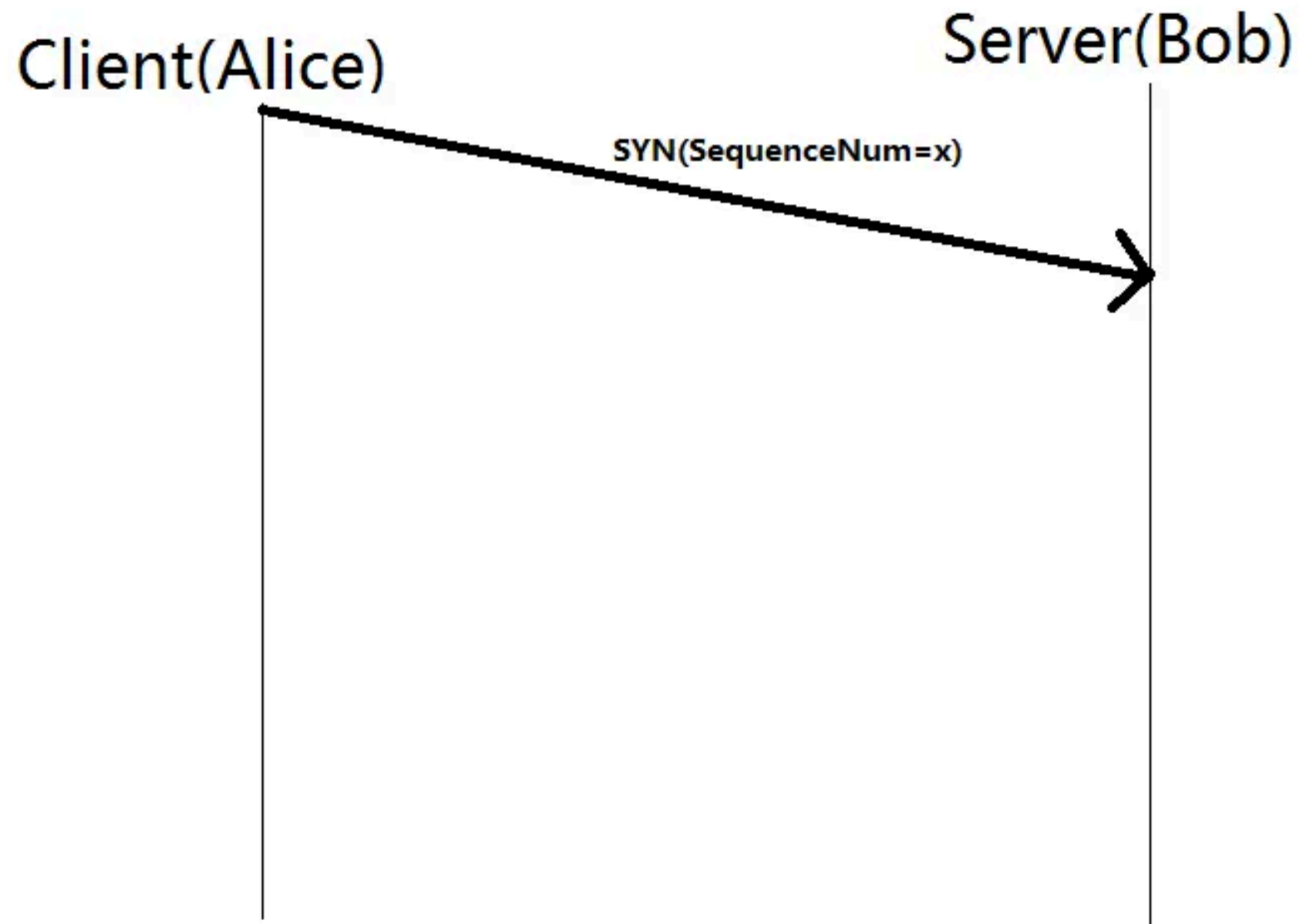
- Connection startup
- Loss recovery mechanisms
- Congestion control
- Interactions with other parts of the network

TCP conceptually

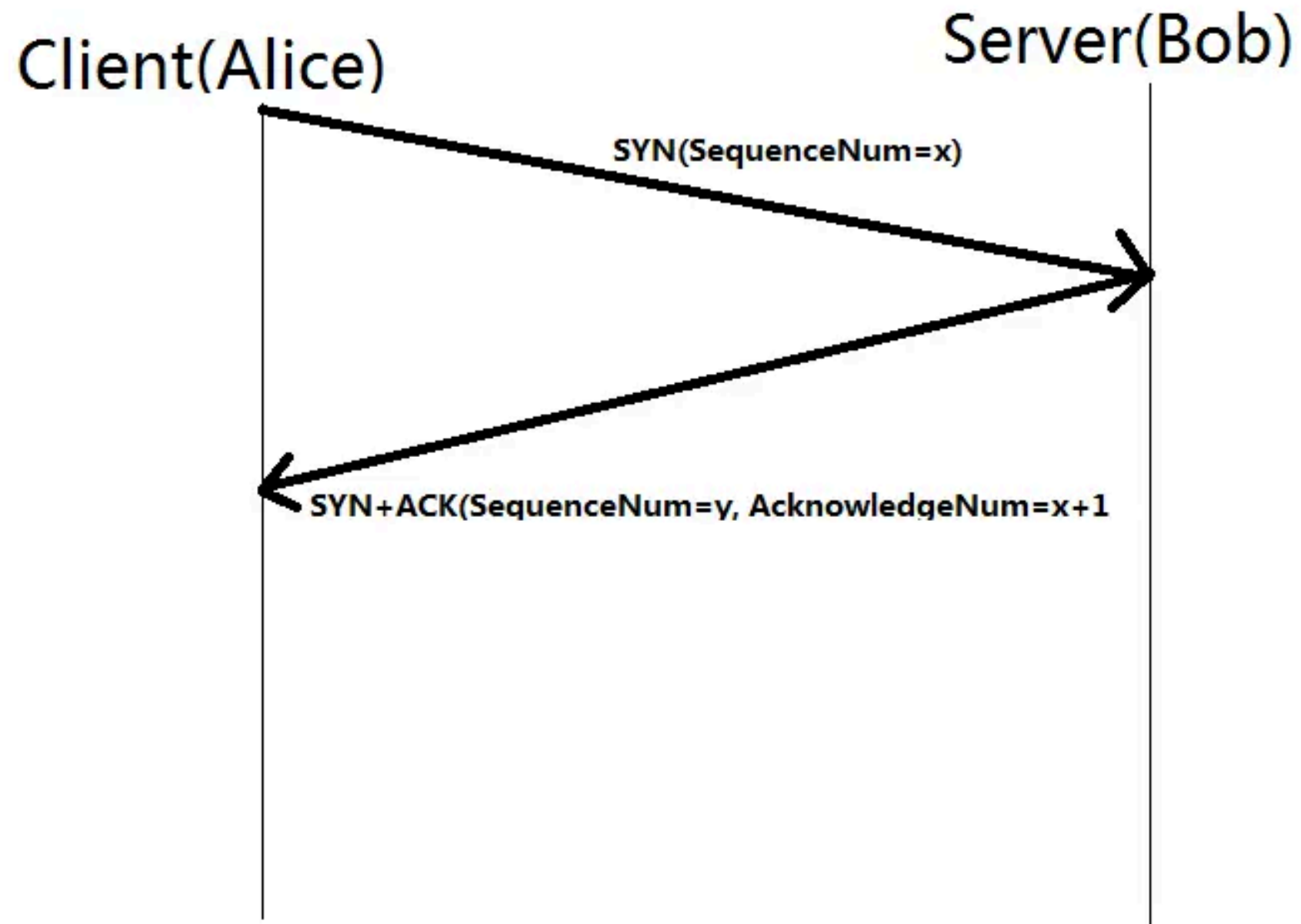
- Connection oriented
 - Have to `connect ( )` before transmitting
- Byte stream based
 - Chopping data into packets is a protocol detail
- Reliable
 - Data will (eventually) arrive in order, or error out
- Adaptable
 - Will attempt to transmit data as fast as possible over any link
- Old
 - [RFC675](#) was first published 51 years ago - lots has happened since!

Connection startup

TCP handshake



TCP handshake



TCP handshake

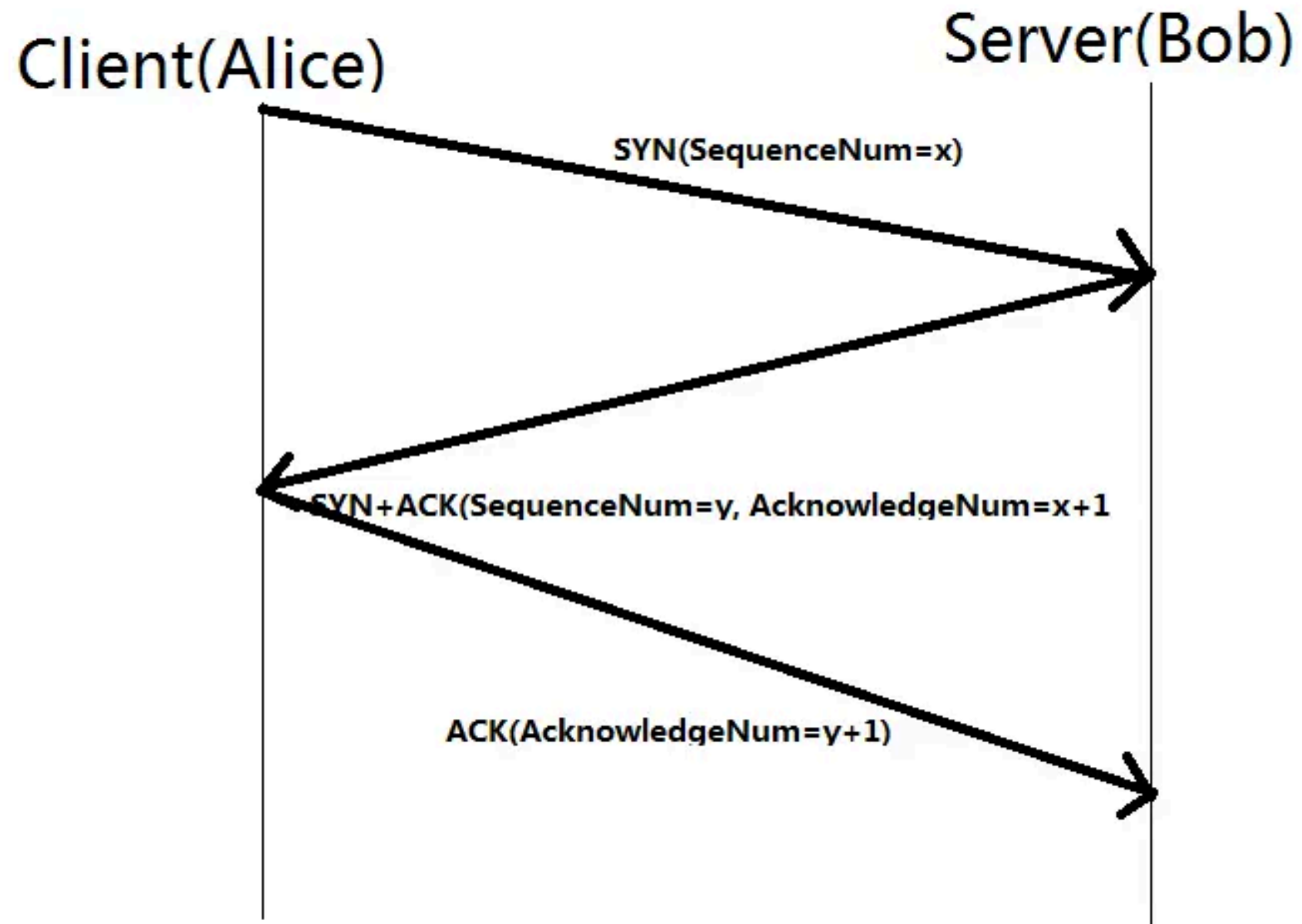


Image source: [https://en.wikipedia.org/wiki/Handshake_\(computing\)](https://en.wikipedia.org/wiki/Handshake_(computing))

Adding TLS

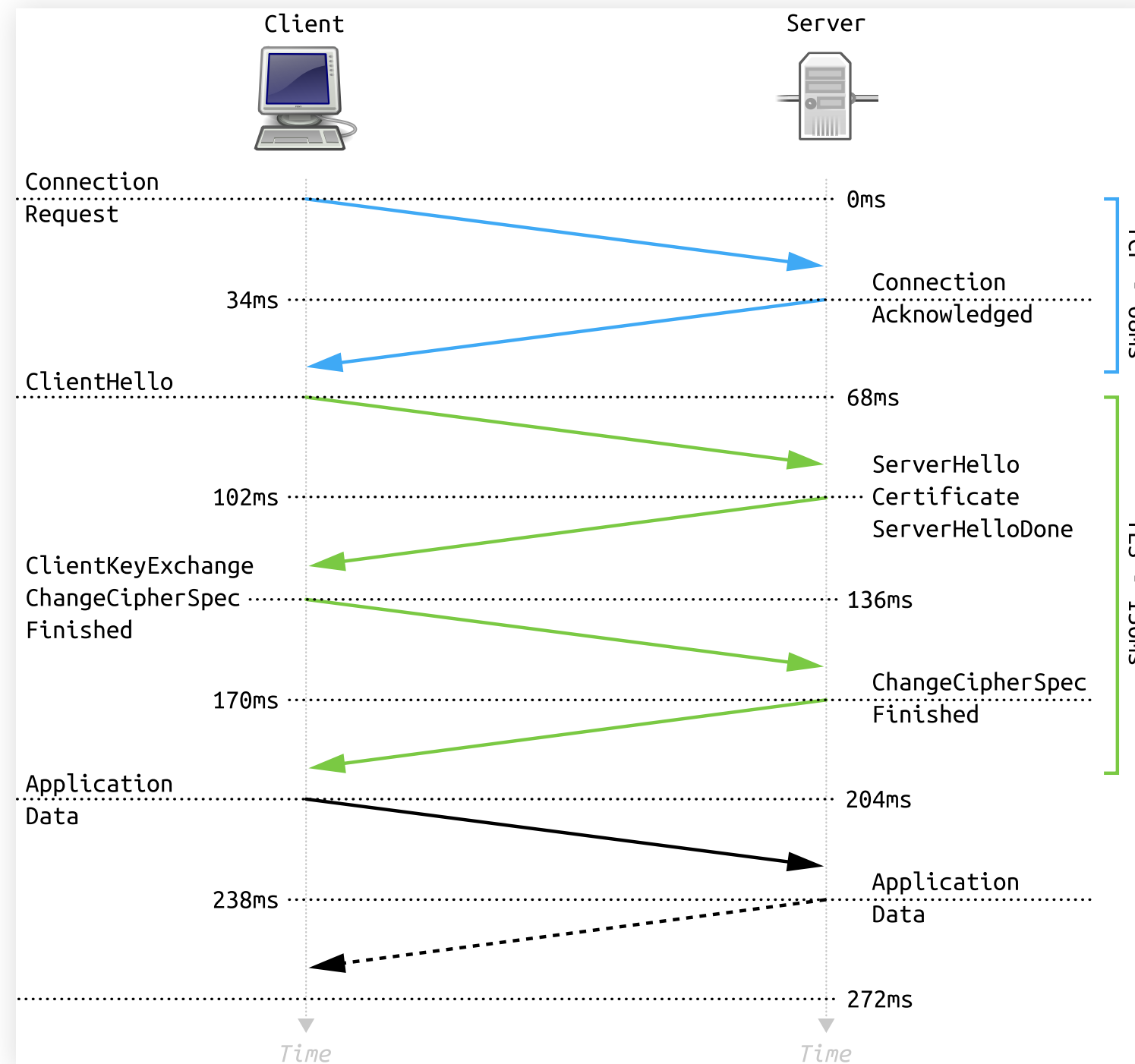


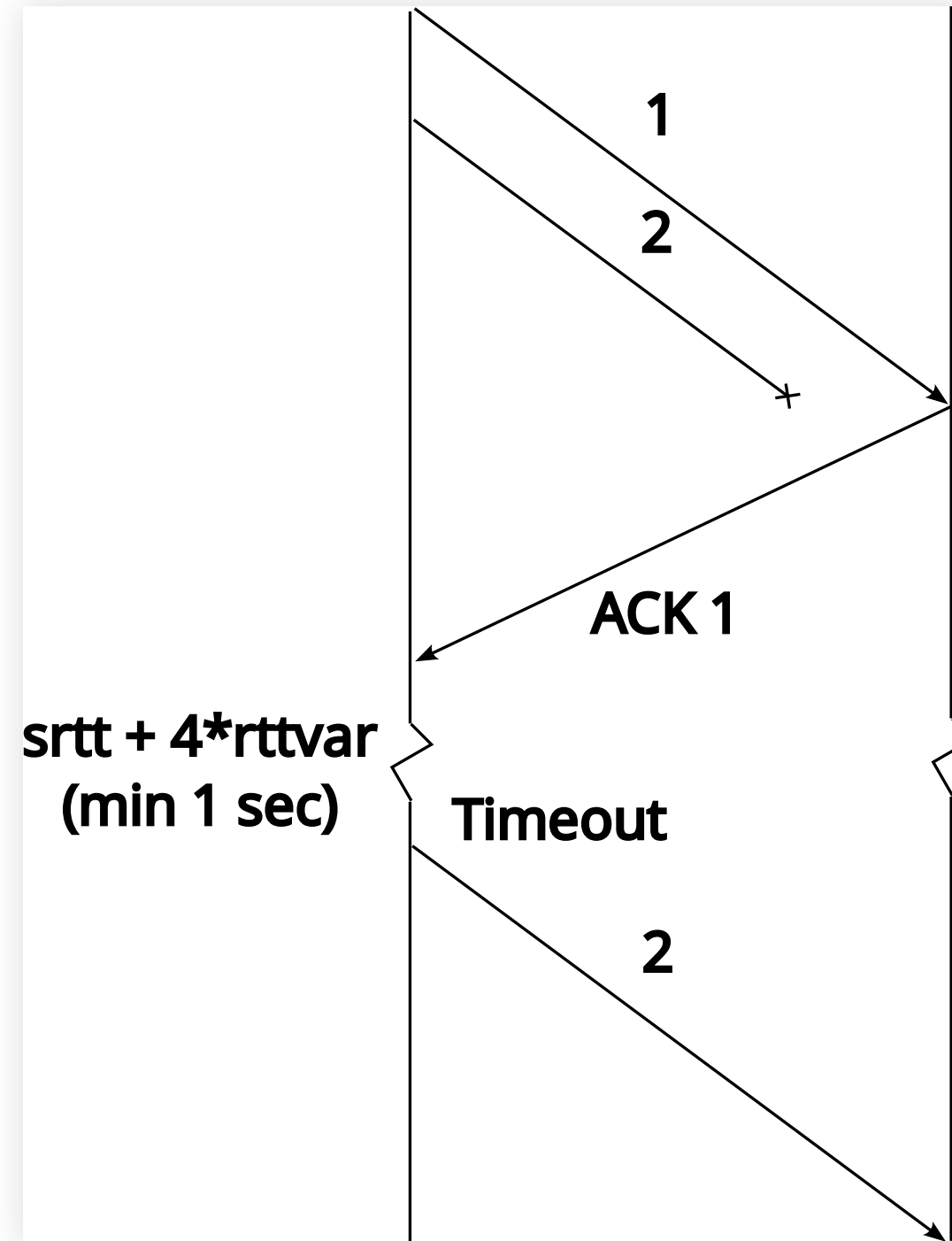
Image source: https://en.wikipedia.org/wiki/Transport_Layer_Security#TLS_handshake

Important points - connection startup

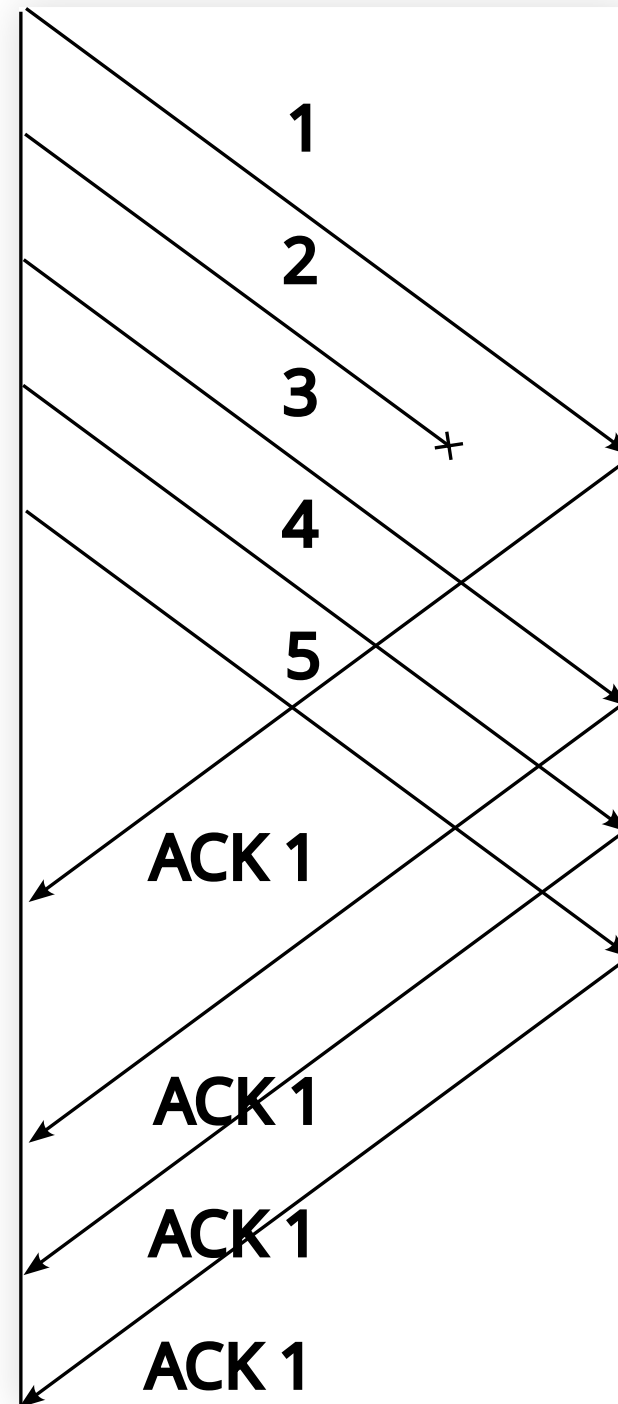
- Small packets
- Multiple round-trips to setup a connection
- Dominated by RTT
- Can take longer than the data transfer!

TCP loss recovery

Retransmission timeout (RTO)

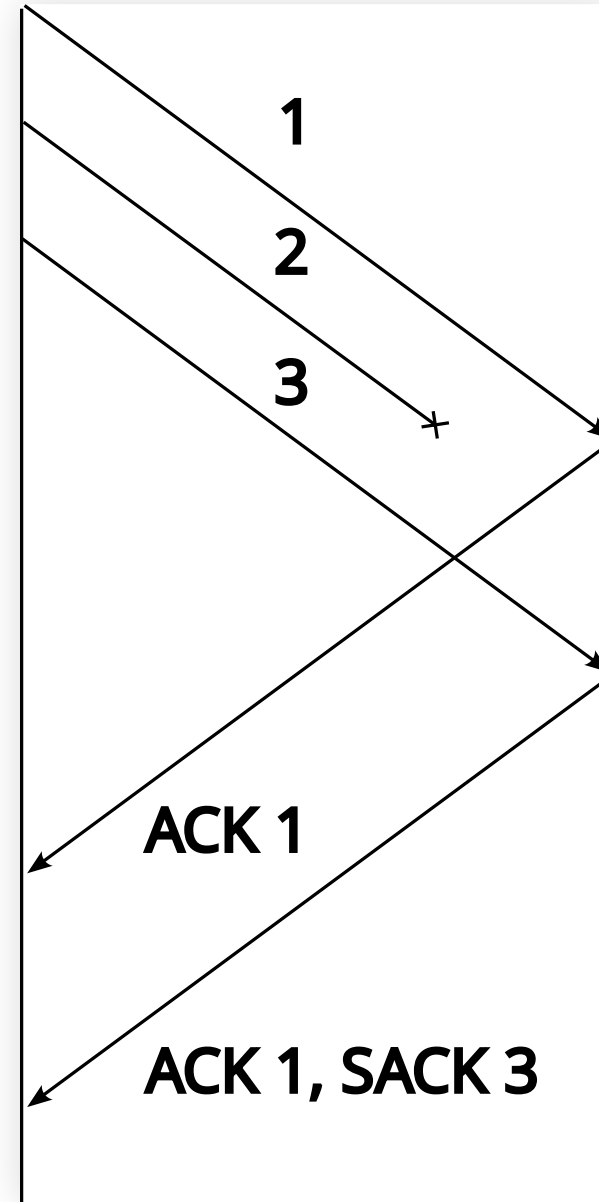


Fast retransmit (DupACK)

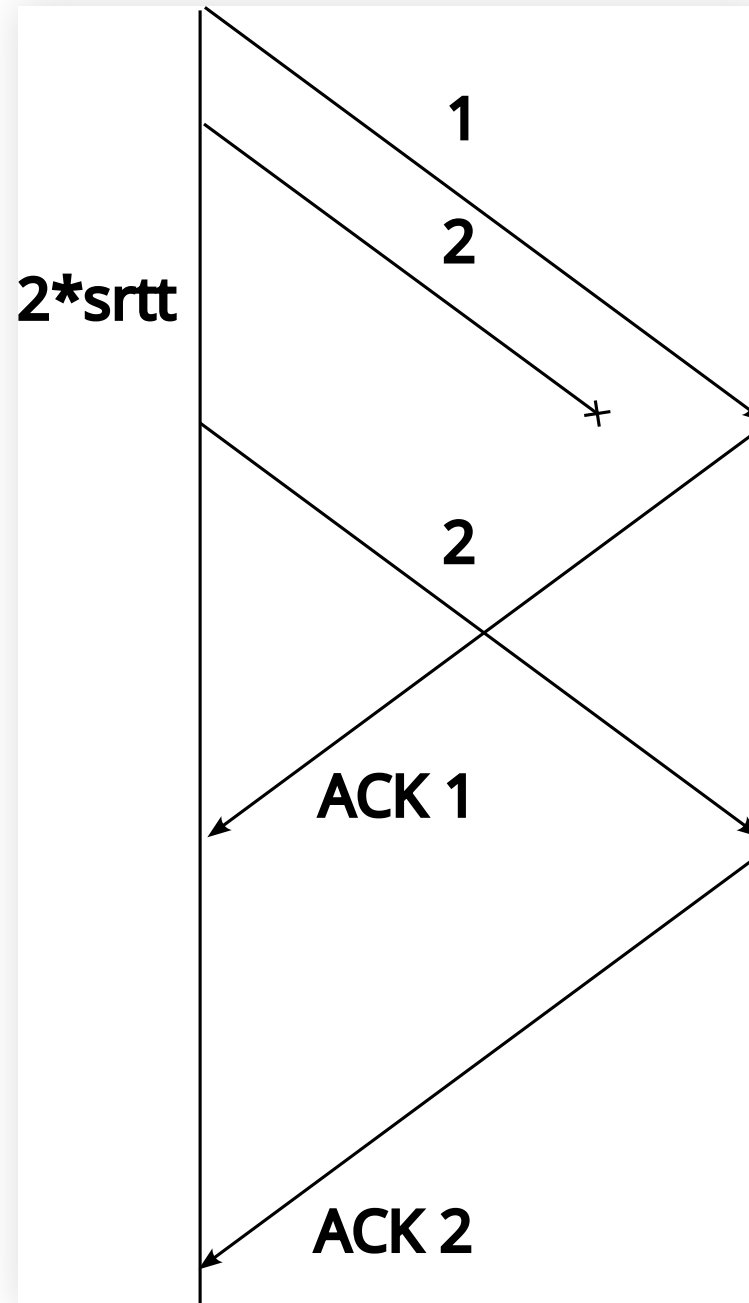


Selective Acknowledgement (SACK and DSACK)

- SACK: [RFC2018](#)
- DSACK: [RFC2883](#)



Tail Loss Probes (TLP)



Shorter timeout, don't reduce window

Recent Acknowledgement (RACK)

- [RFC8985](#)
- Keep a “recent window” based on RTT (`rtt_min/4`)
- Don’t trigger loss recovery if OOO packets arrive within this window
- Significantly improves behaviour when the network reorders packets

Important points - loss recovery

- Several different mechanisms in play
- SACK, RACK and TLP improves behaviour
 - Reordering resilience in particular
- Active research and innovation - moving target!

Congestion control

Congestion control determine TCP behaviour. We'll cover:

- Classic (Reno) TCP
- CUBIC
- BBR

[Wikipedia has a list of other algorithms](#)

Congestion signals

All congestion control algorithms react to one or more signals:

- Packet loss (duplicate ACKs)
- Timeout (RTO)
- ECN notification
- RTT increases

“Slow” start in classic (Reno) TCP

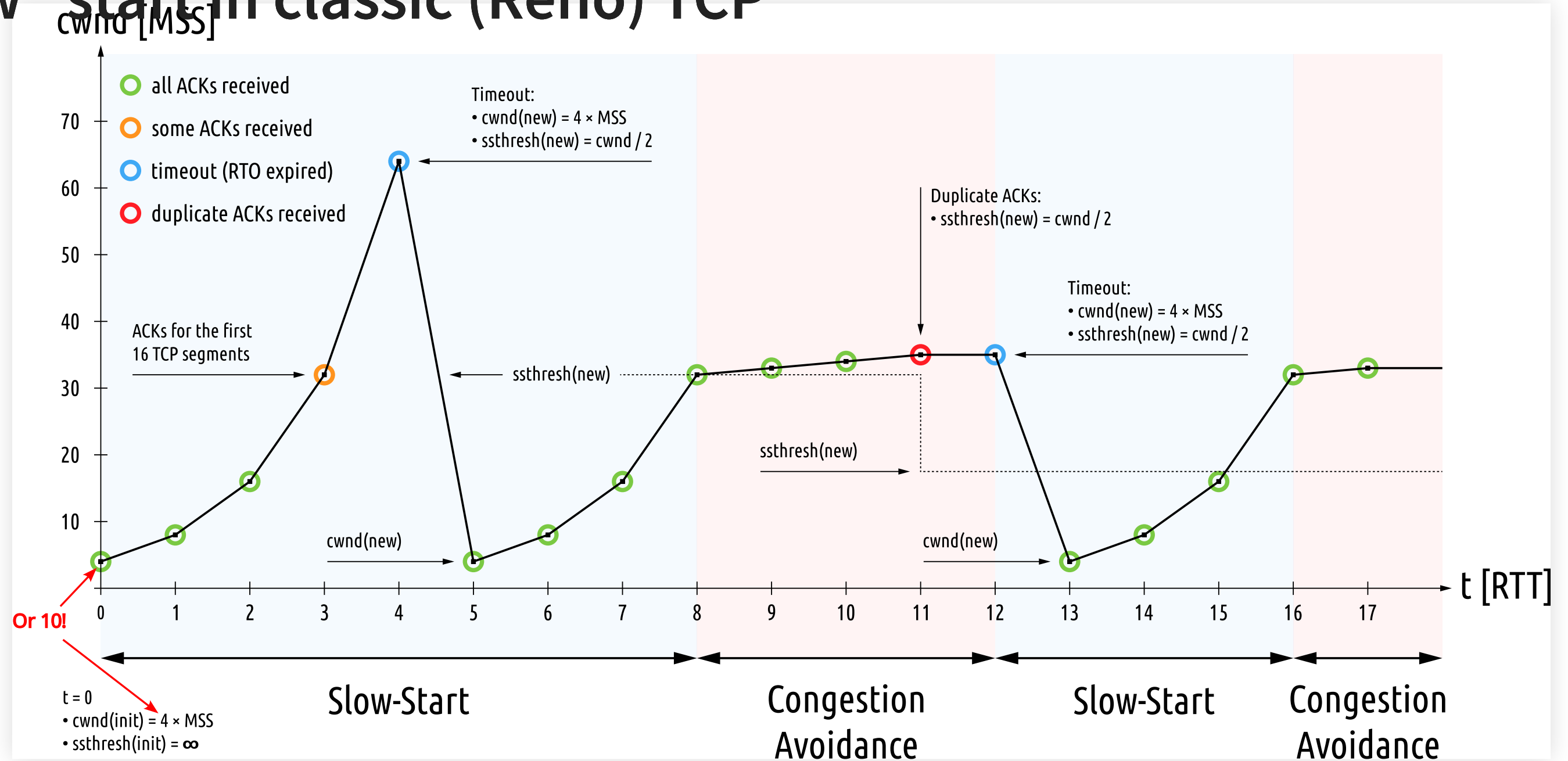
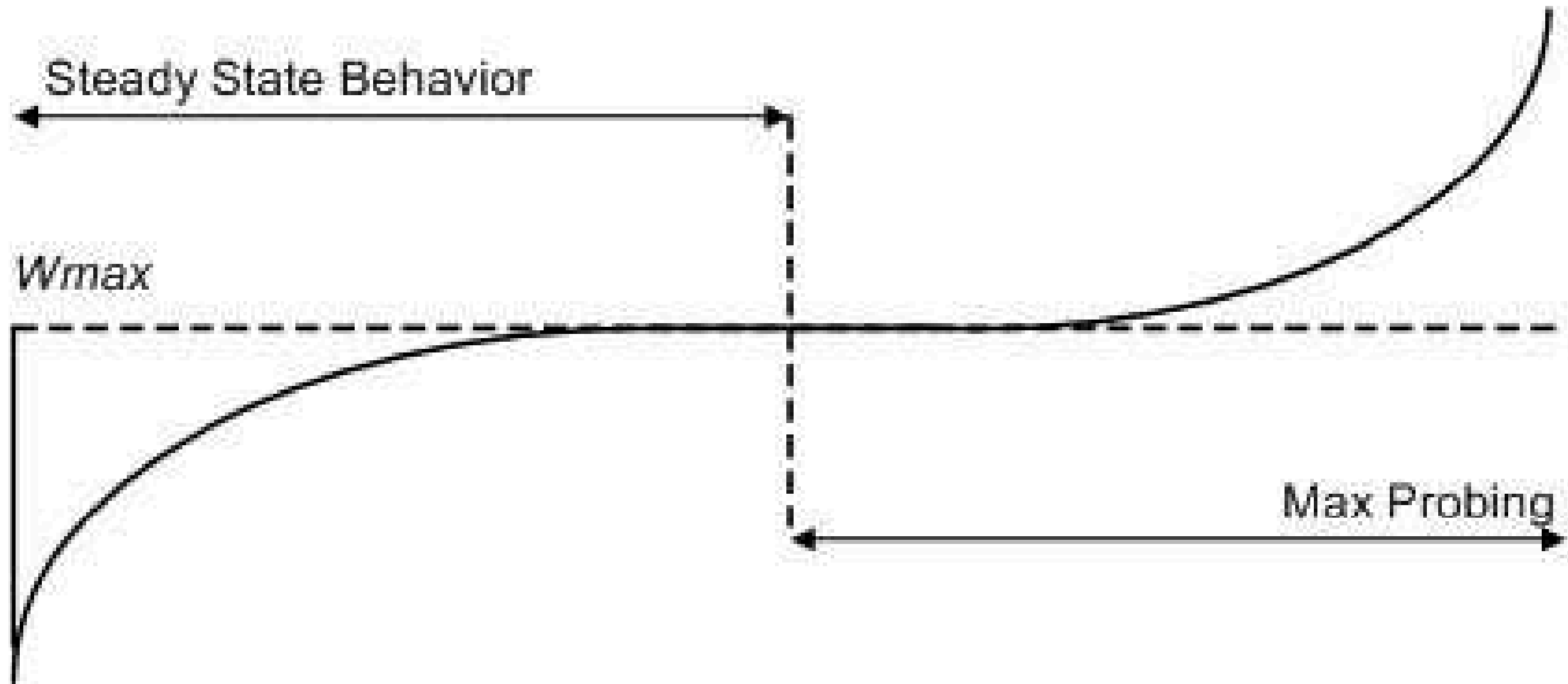


Image source: https://commons.wikimedia.org/wiki/File:TCP_Slow-Start_and_Congestion_Avoidance.svg

Many flows never leave slow start!

TCP CUBIC window function



(b) CUBIC window growth function.

TCP BBR behaviour

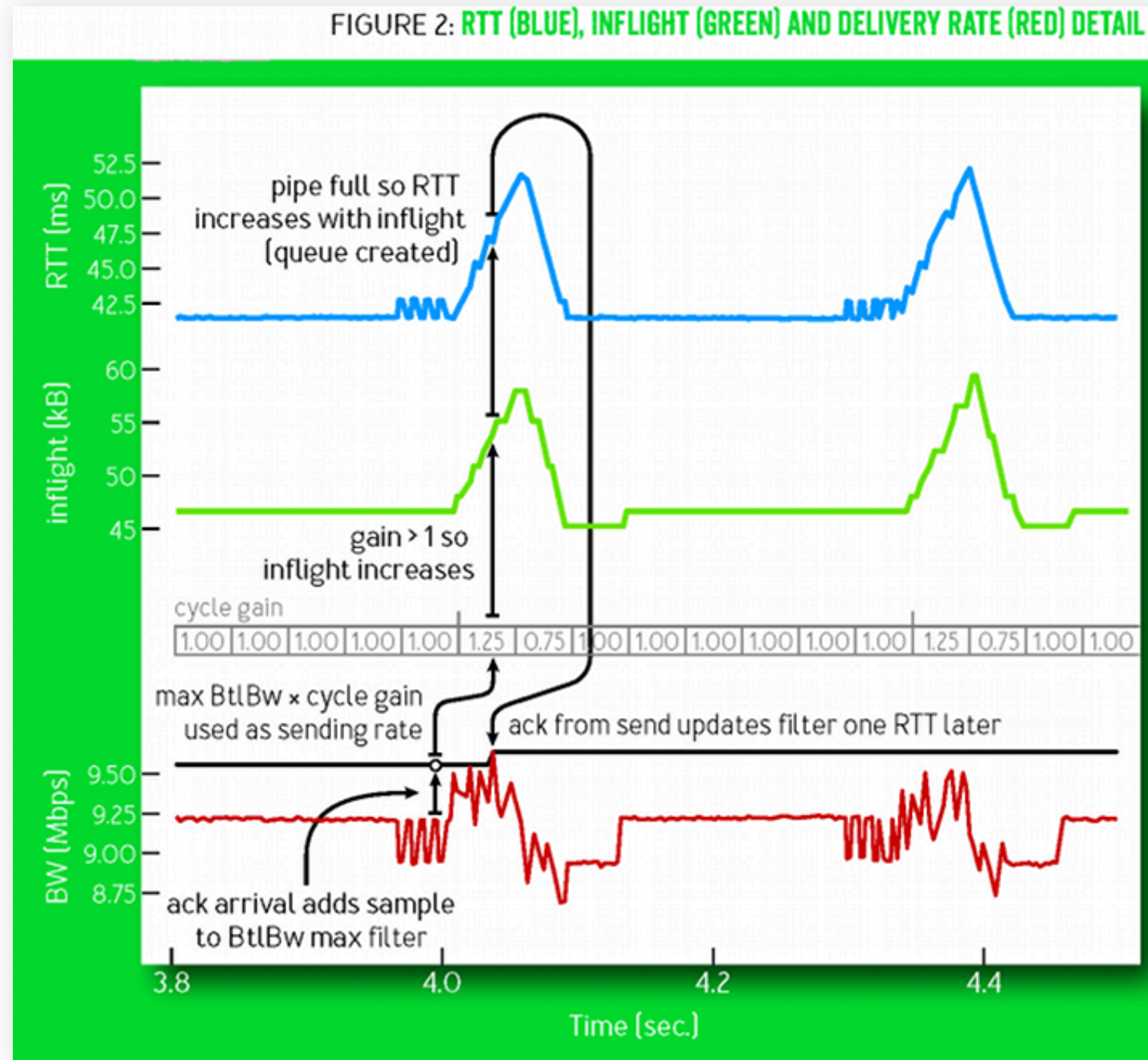


Image source: <https://queue.acm.org/detail.cfm?id=3022184>

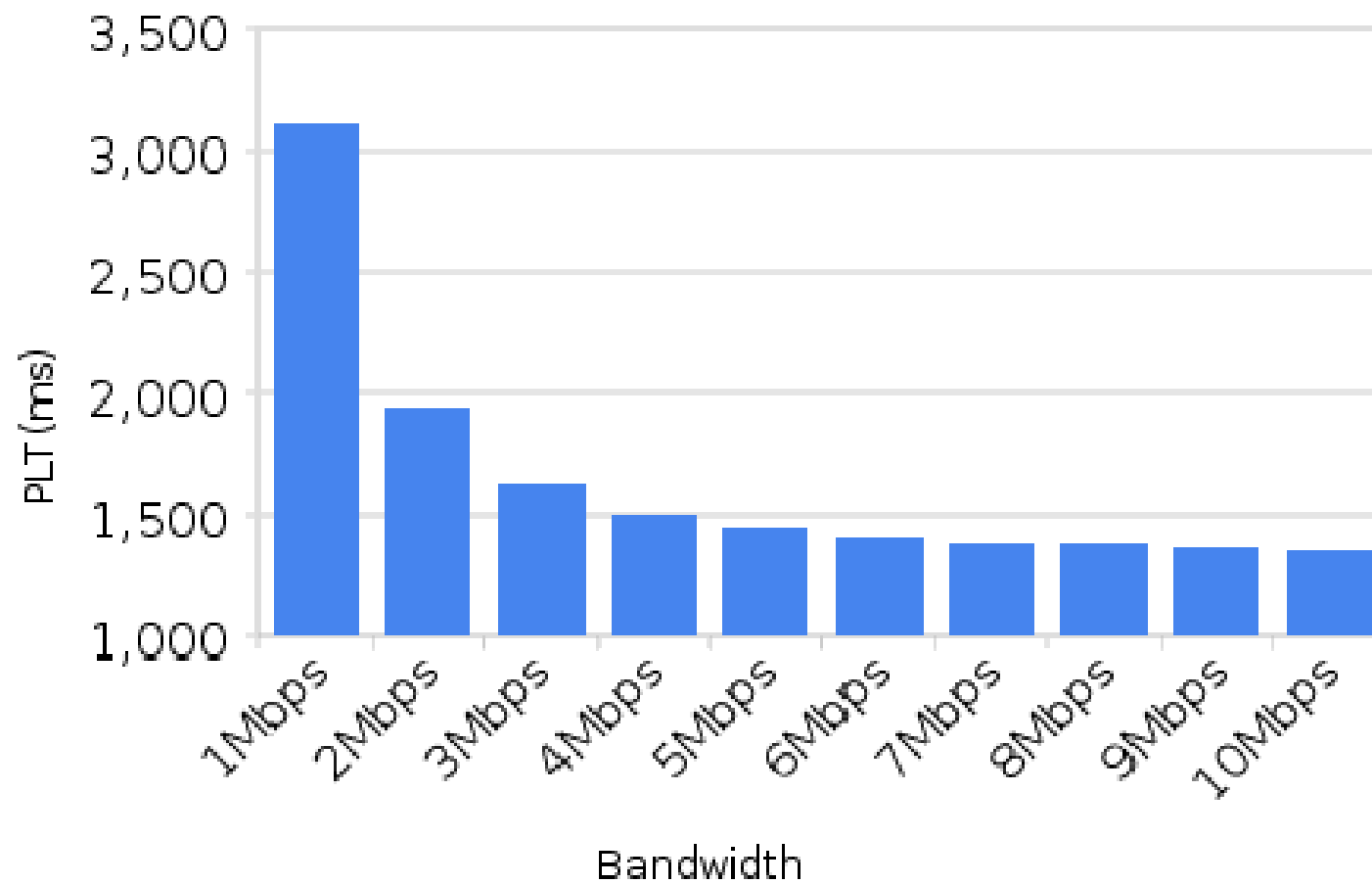
Important points - congestion control

- “Classic” (New) Reno often implicit assumption in literature
- CUBIC actually default, and is more aggressive
- BBR totally different, mainly deployed in the cloud (Google)

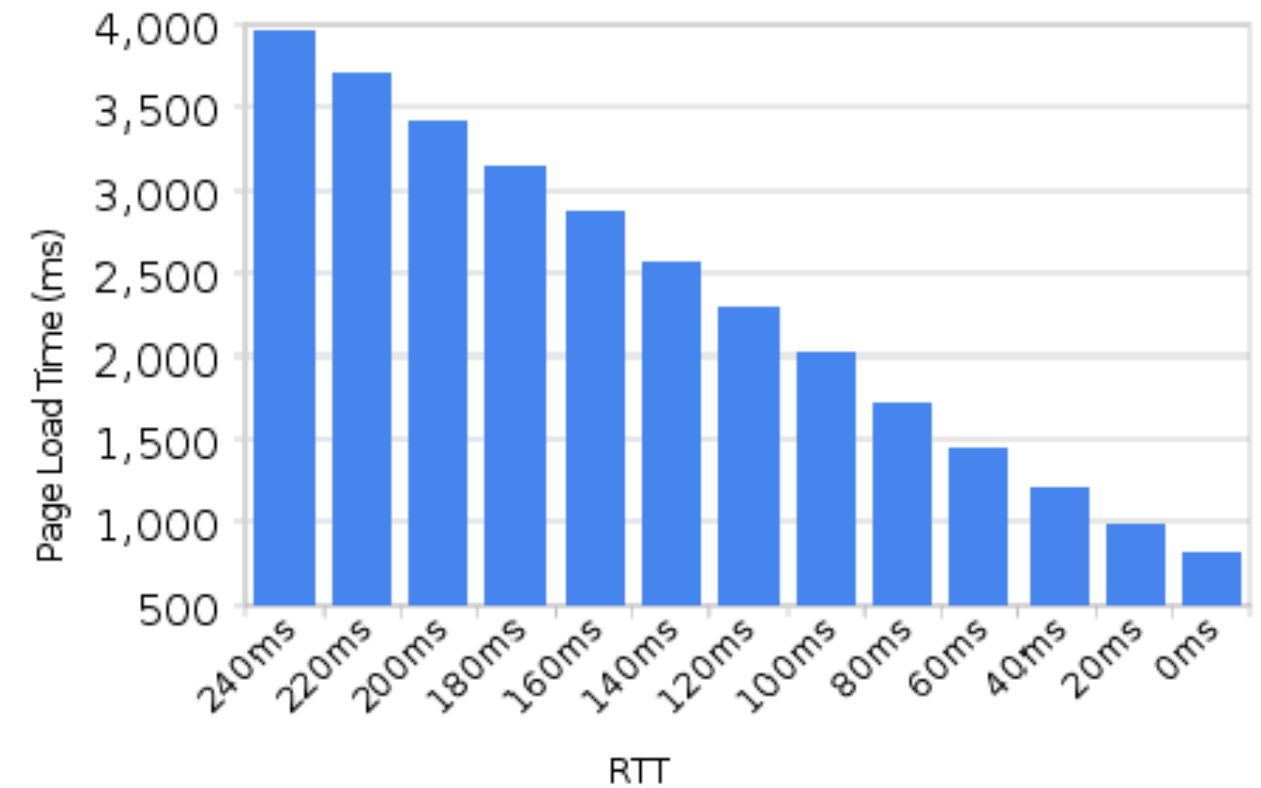
TCP interactions

Latency

Latency per Bandwidth

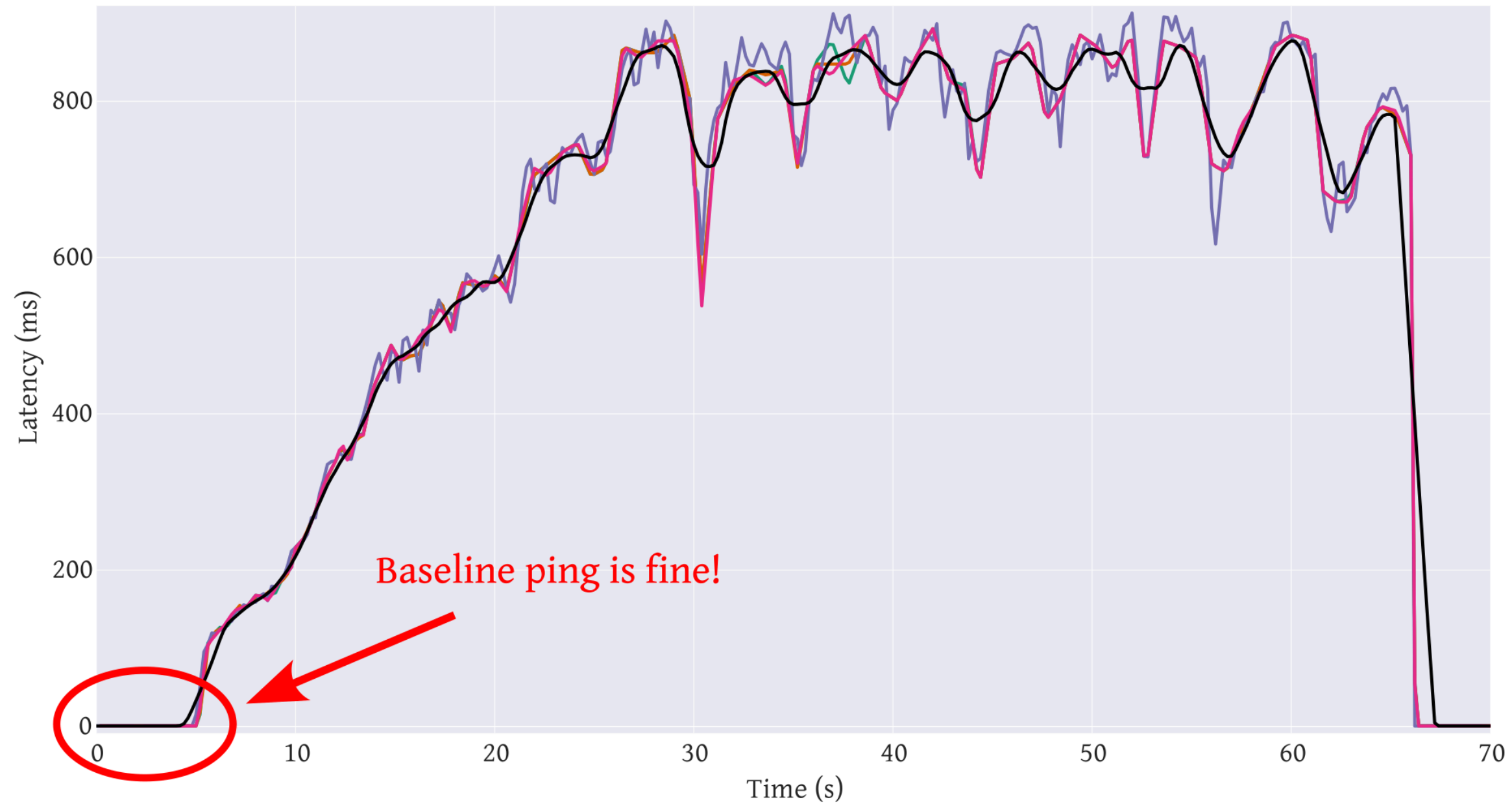


Page Load Time As RTT Decreases



Source: <https://youtu.be/TNBkxA313kk> (from 2011)

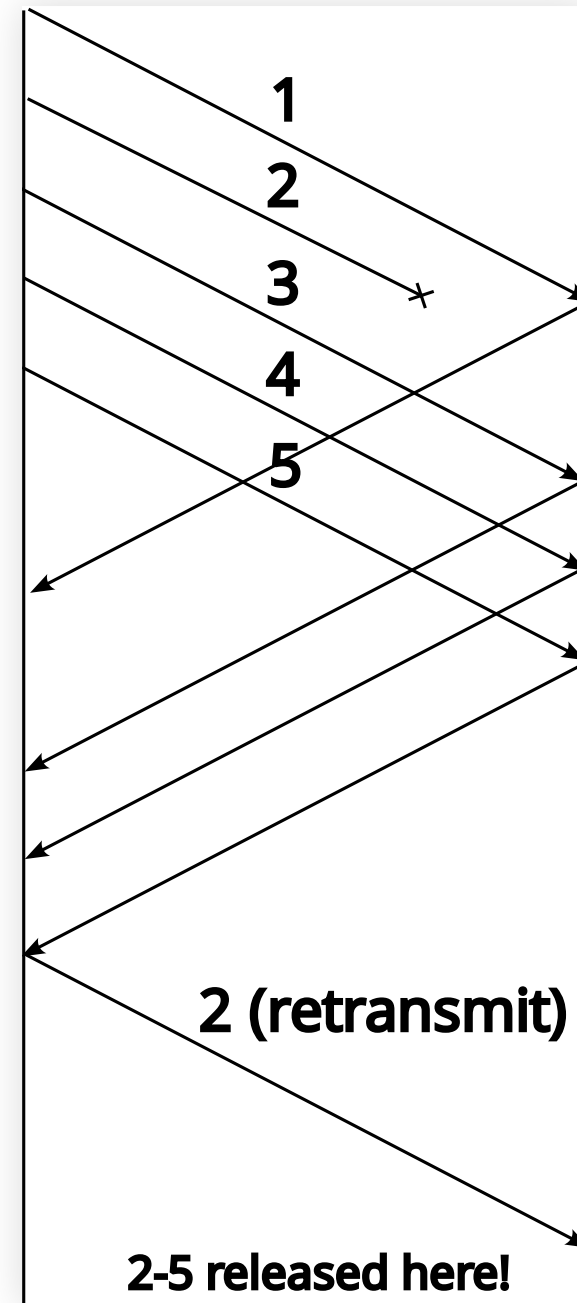
Bufferbloat



Bufferbloat (cont)

- Loss (or ECN mark) is an important signal for TCP!
- Timely signalling required or TCP will keep scaling up
- Active Queue Management (AQM) and Fairness Queueing (FQ) helps
- Excessive link-layer retries add latency

Head of line (HOL) blocking



Also happens when 2 is reordered, not lost.

Important points - TCP interactions

- TCP interacts with other layers in non-obvious ways
- Latency equally important as throughput
- Congestion signalling is important!
 - TCP will fill buffers, leading to bloat
- Hardware offloads and pacing can affect packet sequences

QUIC

What is QUIC?

- QUIC = Quick UDP Internet Connections
- Developed by Google, standardized by IETF ([RFC9000](#))
- Runs over UDP (instead of TCP)
- Combines:
 - Transport (similar to TCP)
 - Security (built-in TLS 1.3)
 - Implemented in user-space
- Foundation of HTTP/3 ([RFC9114](#))

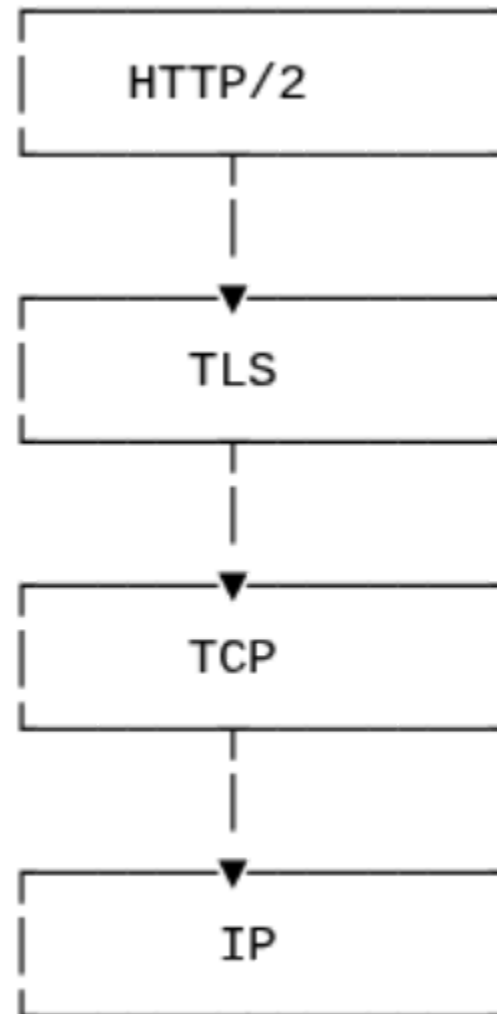
Key selling point:

QUIC reduces latency and improves reliability compared to TCP + TLS

Why does QUIC improve performance?

- Faster connection setup
 - 1-RTT (0-RTT on repeat connections)
- No head-of-line blocking
 - Independent streams within a connection
- Built-in encryption
 - No separate TLS handshake
- Connection migration
 - Survives IP changes (e.g., Wi-Fi → mobile)

Traditional Stack (TCP + TLS)

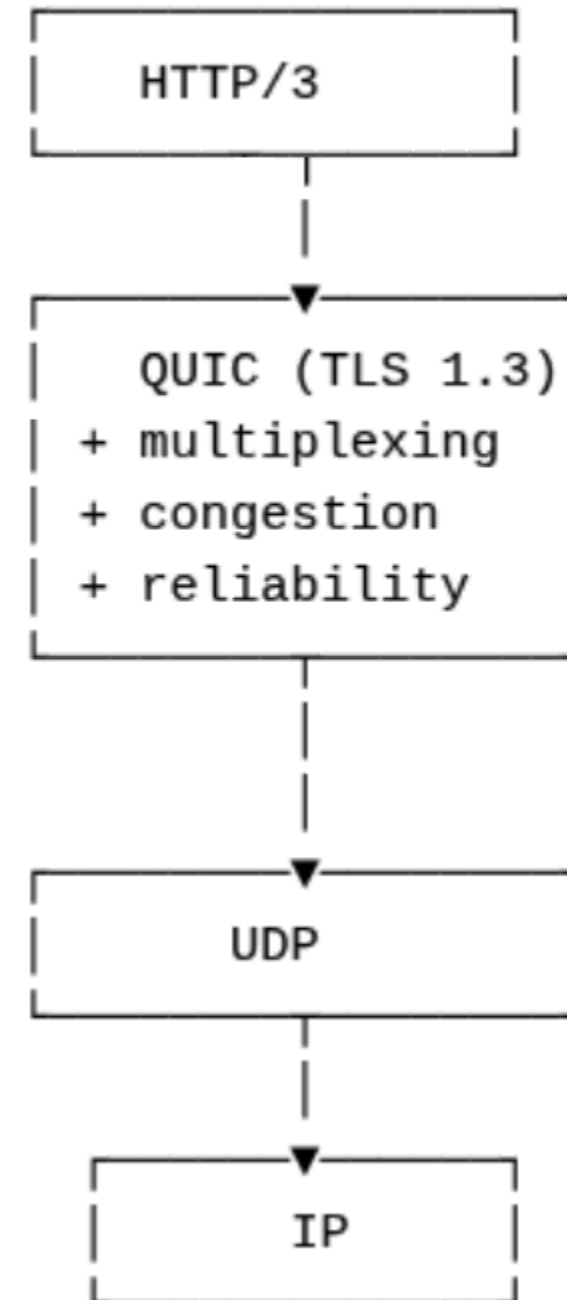


(separate handshake)

(HOL blocking issue)

vs

QUIC Stack (HTTP/3)



Further reading

- Wikipedia: https://en.wikipedia.org/wiki/TCP_congestion_control
- Bufferbloat web site: <https://bufferbloat.net/>
- My own PhD: <https://bufferbloat-and-beyond.net/>
- “It’s the latency, stupid”: <https://www.stuartcheshire.org/rants/Latency.html>
- RFCs can be very readable (links throughout the presentation)